資料結構・課堂 2　Array 陣列　測驗

一、選擇題　（每題 5 分，共 25 分）

1. 陣列存取 `arr[i]` 的時間複雜度是？

(　　) A. O(n)　B. O(log n)　C. O(1)　D. O(n^2)

---

2. `int arr[5]` 的 base address 為 `0x1000`，`sizeof(int) = 4`。`arr[3]` 的記憶體位址是？

(　　) A. `0x1003`　B. `0x1006`　C. `0x100C`　D. `0x1010`

---

3. 在陣列**中間**插入一個元素，時間複雜度是？

(　　) A. O(1)　B. O(log n)　C. O(n)　D. O(n^2)

---

4. `std::vector` 的 `push_back()` 偶爾會變成 O(n)，原因是？

(　　) A. 需要先搜尋是否有重複元素

(　　) B. 容量不足時需重新分配記憶體並複製所有元素

(　　) C. vector 底層是 Linked List

(　　) D. push_back 本來就是 O(n)

---

5. 下列哪個場景**最不適合**使用陣列？

(　　) A. 儲存 50 名學生的成績，需隨機查詢某位同學

(　　) B. 文字編輯器緩衝區（頻繁在任意位置插入/刪除字元）

(　　) C. 儲存一週七天的氣溫紀錄

(　　) D. 實作靜態查找表，索引已知

二、填空題　（每格 4 分，共 24 分）

6. 陣列的記憶體位址計算公式：

`address(arr[i])` = _ _ _ _ _ _ _ _ _ _ + i × _ _ _ _ _ _ _ _ _ _

7. 陣列的優點與限制：

優點 ①：＿＿＿＿＿＿ 存取，時間複雜度 O(1)

優點 ②：Cache ＿＿＿＿＿＿（元素在記憶體中連續排列）

限制 ①：大小 ＿＿＿＿＿＿，無法在執行時動態調整

限制 ②：中間插入/刪除需搬移 O(＿＿＿) 個元素

8. `vector` 容量不足時，會自動將容量 ＿＿＿＿ 倍，

使得 `push_back()` 的**均攤**複雜度仍為 O(＿＿＿)。

三、複雜度表格　（每格 3 分，共 21 分）

請填寫陣列各操作的時間複雜度。

操作	時間複雜度
`arr[i]` 隨機存取	
末尾插入	
末尾刪除	
搜尋（未排序）	
Binary Search（已排序）	
中間插入	
中間刪除	

四、追蹤題　（共 30 分）

9.（15 分） 追蹤 Binary Search 搜尋 `25` 的過程。

陣列：`[ 3, 7, 12, 18, 25, 33, 41, 55 ]`　index：`0 1 2 3 4 5 6 7`

第 1 次：　left = 0，right = 7

mid = ＿＿＿　　arr[mid] = ＿＿＿　　目標 25 比較結果：☐ 找到　☐ 往右（left = ＿＿＿）　☐ 往左（right = ＿＿＿）

第 2 次： left = _ _ _ ，right = _ _ _

mid = _ _ _　　arr[mid] = _ _ _　　目標 25 比較結果：☐ 找到　☐ 往右（left = _ _ _ ）　☐ 往左（right = _ _ _ ）

第 3 次： left = _ _ _ ，right = _ _ _

mid = _ _ _　　arr[mid] = _ _ _　　目標 25 比較結果：☐ 找到！　☐ 繼續...

共比較 _ _ _ 次，時間複雜度 O(_ _ _)

10.（15 分） 在 `arr = [10, 20, 30, 40, 50]` 的 index 2 插入 `99` ，請畫出插入後的陣列並標示搬移方向。

插入前：

```
┌────┬────┬────┬────┬────┐
│ 10 │ 20 │ 30 │ 40 │ 50 │
└────┴────┴────┴────┴────┘
 [0]  [1]  [2]  [3]  [4]
```

插入後（填入每格的值，用箭頭 → 標示哪些元素發生搬移）：

```
┌────┬────┬────┬────┬────┬────┐
│    │    │    │    │    │    │
└────┴────┴────┴────┴────┴────┘
 [0]  [1]  [2]  [3]  [4]  [5]
```

共搬移了 _ _ _ 個元素，這就是 O(n) 的代價。